Lab Title:-NUMBER SYSTEMS, ASCII, TIMESTAMPS AND DATA REPRESENTATION

Student Name:- BADAMASI MUNTAKA MUHAMMAD

Student ID:- 2025FWSD11563

Course Name:- BASIC COMPUTER SKILLS FOR DIGITAL FORENSICS

Instructor Name:- COMMANDANT AMINU IDRIS

Date Of Submission:- 9/08/2026

Executive summary

The lab covers a practical foundational preparation for interpreting digital evidence at the byte, text, time and memory level by understanding how to convert between different number system and also interpreting ASCII value, hashing of text using the echo/echo –n and finally understanding order in which bytes are stored in memory (ENDIANNESS)

Lab Objectives

The objectives of the lab is to understand how to convert from one number system to another, ASCII, Timestamp and data representation which is a very important aspect in interpreting digital evidence.

Tools and Resources Used

Kali linux:- A penetration testing platform used form security assessment

Python3:- Python is an interpreted, interactive, object-oriented programming language that combines remarkable power with very clear syntax.

Xxd:- make a hex dump or do the reverse.

Md5/sha256:-

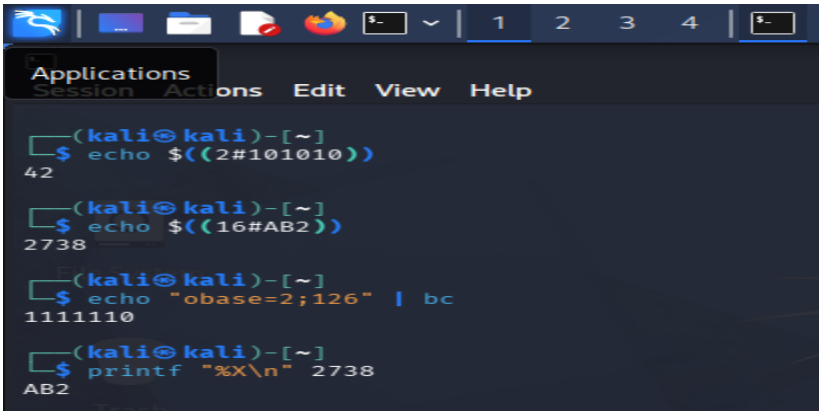
Date:- print or set the system date and time. It Display date and time in the given format.

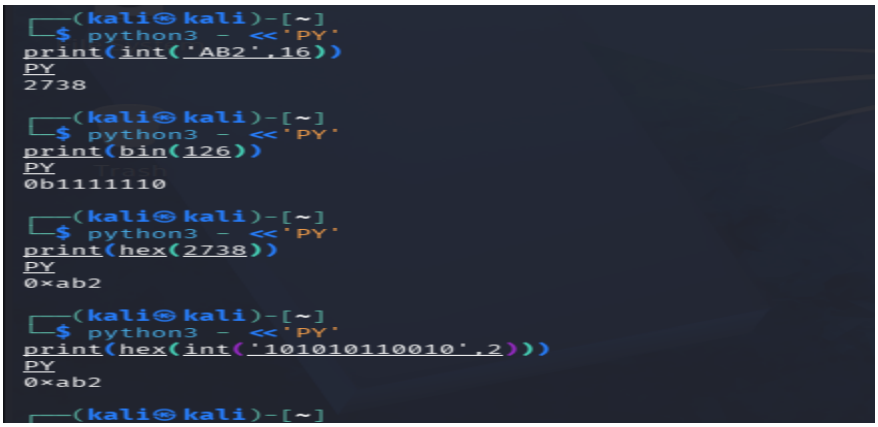
Methology

I navigate to the kali linux command prompt and execute different number system, timestamp, and hashing command.

Screenshot and Evidence

TASK A:- NUMBER SYSTEM AND MANUAL CONVERSION

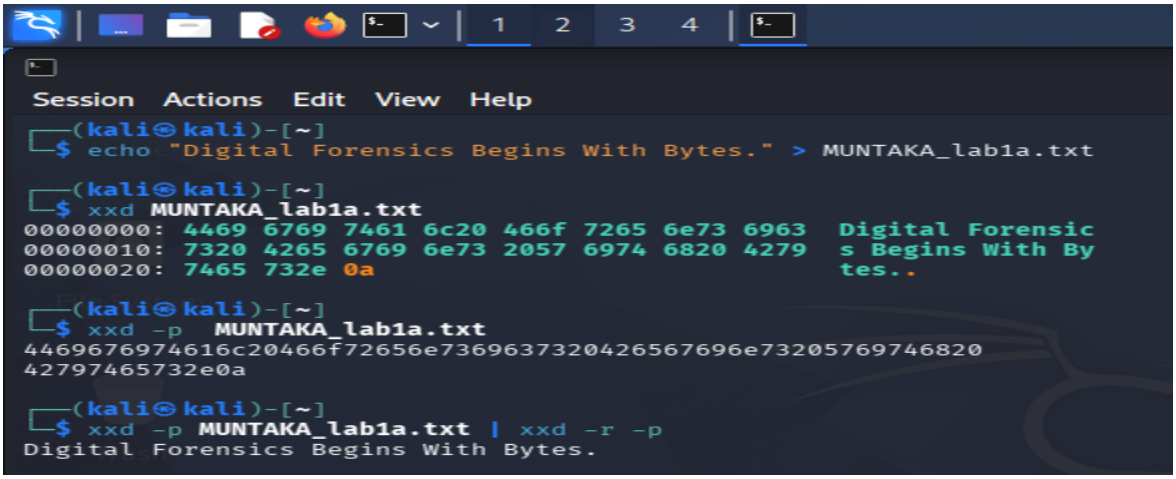




<table>
<tr><th>EVIDENCE VALUE</th><th>GIVEN BASE</th><th>MANUAL ANSWER</th><th>COMMAND USED</th></tr>
<tr><td>101010</td><td>Binary</td><td>
<table>
<tr><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td></tr>
<tr><td>5</td><td>4</td><td>3</td><td>2</td><td>1</td><td>0</td></tr>
<tr><td>2^5</td><td>2^4</td><td>2^3</td><td>2^2</td><td>2^1</td><td>2^0</td></tr>
<tr><td>32</td><td>16</td><td>8</td><td>4</td><td>2</td><td>1</td></tr>
</table>
= 42
</td><td>echo $((2#101010))</td></tr>
<tr><td>AB2</td><td>Hexadecimal</td><td>Where A= 10, B= 11
<table>
<tr><td>A</td><td>B</td><td>2</td></tr>
<tr><td>2</td><td>1</td><td>0</td></tr>
<tr><td>$10\ X\ 16^2$</td><td>$11\ X\ 16^1$</td><td>$2\ X\ 16^0$</td></tr>
</table>
</td><td>echo $((16#AB2))</td></tr>
</table>

<table>
<tr>
<td></td>
<td></td>
<td>
<table>
<tr><td>2560</td><td>176</td><td>2</td></tr>
</table>
= 2738
</td>
<td></td>
</tr>
<tr>
<td>126</td>
<td>Decimal</td>
<td>Where R = reminder
<table>
<tr><td>126</td><td>R</td></tr>
<tr><td>126 ÷ 2 = 63</td><td>0</td></tr>
<tr><td>63 ÷ 2 = 31</td><td>1</td></tr>
<tr><td>31 ÷ 2 = 15</td><td>1</td></tr>
<tr><td>15 ÷ 2 = 7</td><td>1</td></tr>
<tr><td>7 ÷ 2 = 3</td><td>1</td></tr>
<tr><td>3 ÷ 2 = 1</td><td>1</td></tr>
<tr><td>1 ÷ 2 = 0</td><td>1 ↑</td></tr>
</table>
126 = 1111110
</td>
<td>echo "obase=2;126" | bc</td>
</tr>
<tr>
<td>2738</td>
<td>Decimal</td>
<td>Where R = reminder
<table>
<tr><td>2738</td><td>R</td></tr>
<tr><td>2738 ÷ 16 = 171</td><td>2</td></tr>
<tr><td>171 ÷ 16 = 10</td><td>11 = B</td></tr>
<tr><td>10 ÷ 16 = 0</td><td>10 = A ↑</td></tr>
</table>
2738 = AB2
</td>
<td>printf "%X\n" 2738</td>
</tr>
<tr>
<td>101010110010</td>
<td>binary</td>
<td>101010110010
<table>
<tr><td>1010</td><td>1011</td><td>0010</td></tr>
<tr><td>10(A)</td><td>11(B)</td><td>2</td></tr>
</table>
101010110010 = AB2
</td>
<td>python3 - <<'PY'
print(hex(int('101010110010',2)))
PY</td>
</tr>
</table>

TASK B:- ASCII, UTF-8 AND HEX EVIDENCE STRINGS



TASK C:- EPOCH TIME AND FORENSIC TIMESTAMPS

```
(kali㉿kali)-[~]
$ date -u -d @1643643491
Mon Jan 31 03:38:11 PM UTC 2022

(kali㉿kali)-[~]
$ date -u -d @1675008832
Sun Jan 29 04:13:52 PM UTC 2023

(kali㉿kali)-[~]
$ python3 - <<'PY'
from datetime import datetime, timezone

unix_epoch = datetime(1970, 1, 1, tzinfo=timezone.utc)
mac_epoch = datetime(2001, 1, 1, tzinfo=timezone.utc)

print(int((mac_epoch - unix_epoch).total_seconds()))

mac_value = 725846400
unix_value = mac_value + 978307200

print(datetime.fromtimestamp(unix_value, tz=timezone.utc))
PY
978307200
2024-01-02 00:00:00+00:00

(kali㉿kali)-[~]
$ date +%s
1786219154

(kali㉿kali)-[~]
$
```

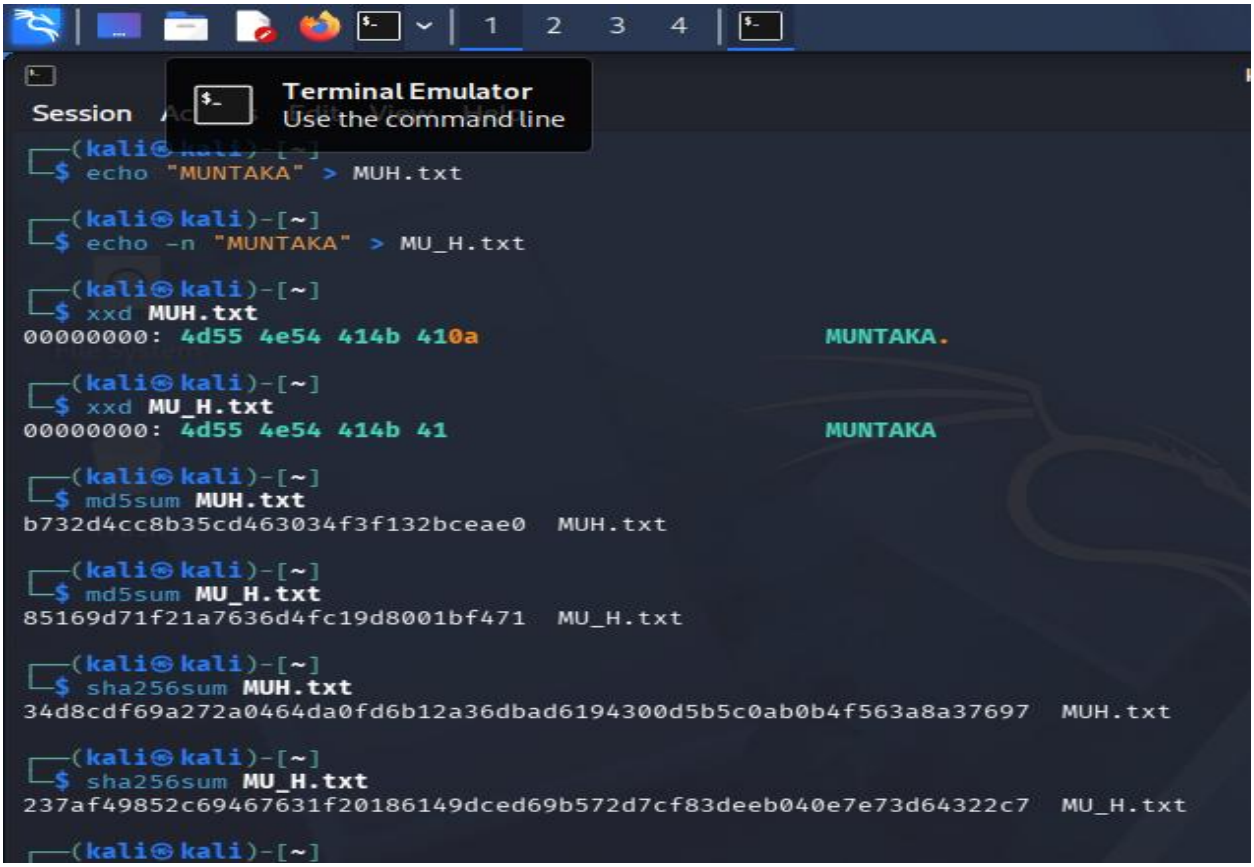
TIMESTAMP VALUE	FORMAT	CONVERSION COMMAND	READABLE UTC RESULT
1643643491	Unix seconds	date -u -d @1643643491	Sun jan 31 03:38:52 PM UTC 2022
1675008832	Unix seconds	date -u -d @1675008832	Sun jan 29 04:13:52 PM UTC 2023
725846400	MAC absolute time seconds	python3 - <<'PY' from datetime import datetime, timezone unix_epoch = datetime(1970, 1, 1, tzinfo=timezone.utc) mac_epoch = datetime(2001, 1, 1, tzinfo=timezone.utc) print(int((mac_epoch - unix_epoch).total_seconds())) mac_value = 725846400 unix_value = mac_value + 978307200 print(datetime.fromtimestamp(unix_value, tz=timezone.utc)) PY	2024-01-02 00:00:00+00:00
Current time	Unix seconds	date +%s	1786219154

Question

1. Explain why 0A may appear at the end of output when echo is used without -n.

Answer

TASK D - HASHING AND THE NEWLINE PROBLEM



Questions

1. Explain why the hash values differ.

Answer:- the reason why the hash values differ is because the “echo” command uses a new line character and when hashing file created using the “echo” command everything will hashed including the new line character.

2. State which command you would use if you needed to hash exactly the visible characters only.

Answer:- I will used the “echo -n” command to hash since It does not include a new line character and since I will be hashing exactly the visible character only.

TASK E - ENDIANNESS AND DATA STORED IN MEMORY

```
┌──(kali㉿kali)-[~]
└─$ python3 - <<'PY'
raw = bytes.fromhex('01000000')
print('Raw bytes:', raw.hex())
print('Interpreted as little-endian:', int.from_bytes(raw, 'little'))
print('Interpreted as big-endian    :', int.from_bytes(raw, 'big'))
PY

Raw bytes: 01000000
Interpreted as little-endian: 1
Interpreted as big-endian    : 16777216
```

```
┌──(kali㉿kali)-[~]
└─$ python3 - <<'PY'
raw = bytes.fromhex('FF000000')
print('Raw bytes:', raw.hex())
print('Interpreted as little-endian:', int.from_bytes(raw, 'little'))
print('Interpreted as big-endian    :', int.from_bytes(raw, 'big'))
PY

Raw bytes: ff000000
Interpreted as little-endian: 255
Interpreted as big-endian    : 4278190080

┌──(kali㉿kali)-[~]
└─$ python3 - <<'PY'
raw = bytes.fromhex('00000100')
print('Raw bytes:', raw.hex())
print('Interpreted as little-endian:', int.from_bytes(raw, 'little'))
print('Interpreted as big-endian    :', int.from_bytes(raw, 'big'))
PY

\Raw bytes: 00000100
Interpreted as little-endian: 65536
Interpreted as big-endian    : 256

┌──(kali㉿kali)-[~]
```

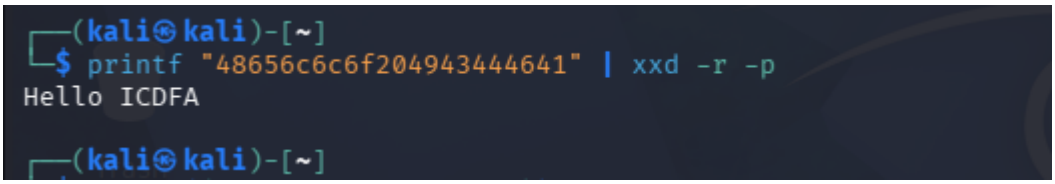
```
1  2  3  4

Session  Actions  Edit  View  Help

┌──(kali㉿kali)-[~]
└─$ python3 - <<'PY'
raw = bytes.fromhex('78563412')
print('Raw bytes:', raw.hex())
print('Interpreted as little-endian:', int.from_bytes(raw, 'little'))
print('Interpreted as big-endian    :', int.from_bytes(raw, 'big'))
PY

Raw bytes: 78563412
Interpreted as little-endian: 305419896
Interpreted as big-endian    : 2018915346
```

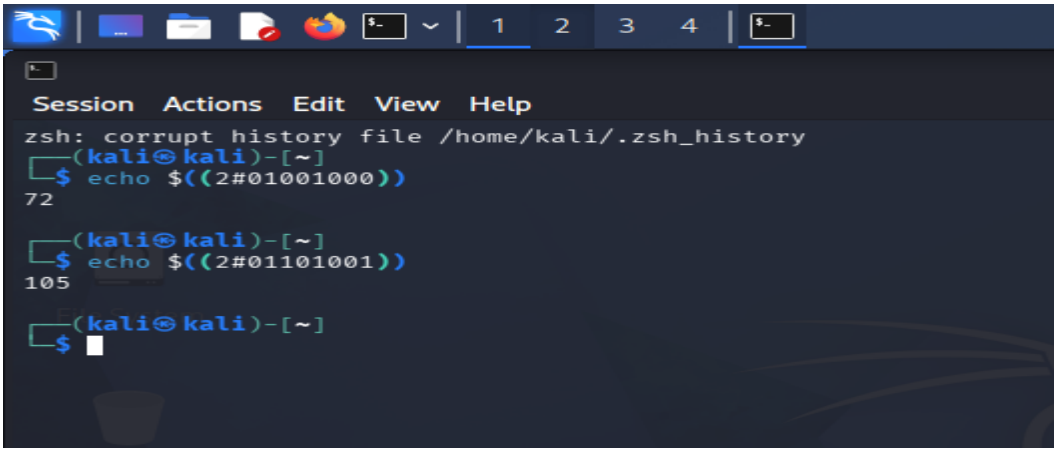
RAW HEX BYTES	LITTLE-ENDIAN DECIMAL	BIG-ENDIAN DECIMAL	COMMAND USED	EXPLANATION
01000000	1	16777216	python3 - <<'PY' raw = bytes.fromhex('01000000')	The Little-endian of the RAW HEX

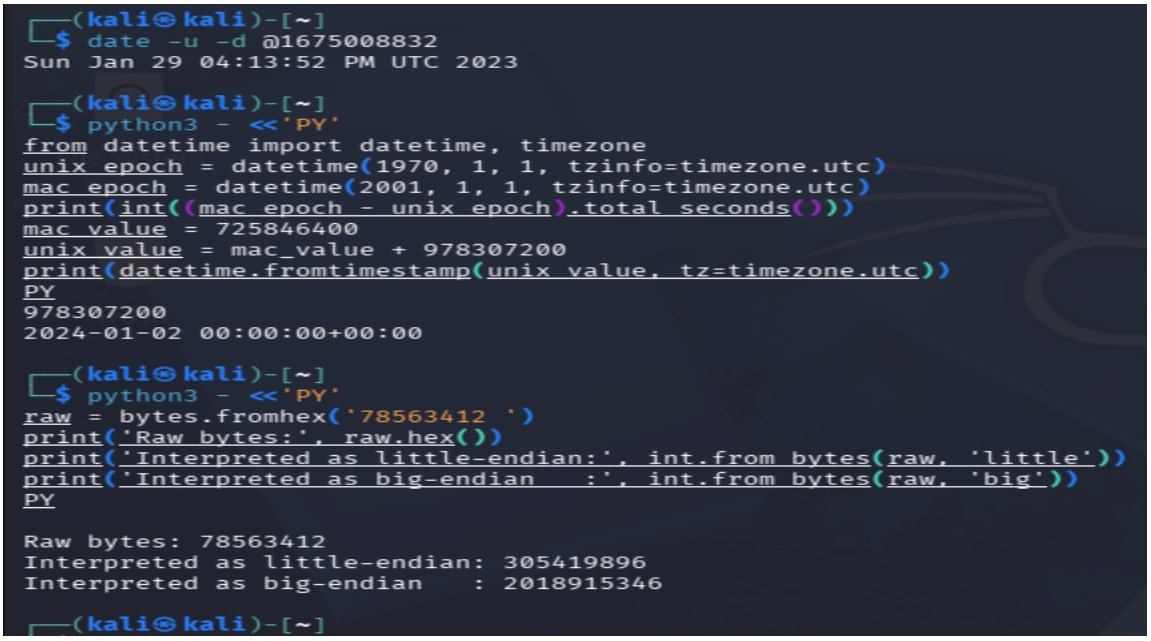
			print('Raw bytes:', raw.hex()) print('Interpreted as little-endian:', int.from_bytes(raw, 'little')) print('Interpreted as big-endian :', int.from_bytes(raw, 'big')) PY	BYTES reads the least significant byte first, while big-endian reads the most significant byte first.
FF000000	255	4278190080	python3 - <<'PY' raw = bytes.fromhex(' FF000000') print('Raw bytes:', raw.hex()) print('Interpreted as little-endian:', int.from_bytes(raw, 'little')) print('Interpreted as big-endian :', int.from_bytes(raw, 'big')) PY	The Little-endian of the RAW HEX BYTES reads the least significant byte first, while big-endian reads the most significant byte first.
00000100	65536	256	python3 - <<'PY' raw = bytes.fromhex('00000100') print('Raw bytes:', raw.hex()) print('Interpreted as little-endian:', int.from_bytes(raw, 'little')) print('Interpreted as big-endian :', int.from_bytes(raw, 'big')) PY	The Little-endian of the RAW HEX BYTES reads the least significant byte first, while big-endian reads the most significant byte first.
78563412	305419896	2018915346	python3 - <<'PY' raw = bytes.fromhex('78563412') print('Raw bytes:', raw.hex()) print('Interpreted as little-endian:', int.from_bytes(raw, 'little'))	The Little-endian of the RAW HEX BYTES reads the least significant byte first, while big-endian reads the most significant byte first.

			print('Interpreted as big-endian :', int.from_bytes(raw, 'big')) PY	

TASK F - MINI FORENSIC INTERPRETATION CASE







Questions

1. State what each evidence item represents after decoding.

Answer

EVIDENCE TABLE
EVID-001 reperesent a hexadecimal encoded ASCII message
EVID-002 reperesent a binary encoded ASCII message
EVID-003 reperesent a unix timestamp
EVID-004 represent a MAC absolute time value
EVID-005 represent the bytes in which order are stored in memory (ENDIANNESS)

2. For timestamp values, state the readable UTC time and the method used.

Answer

A) The readable UTC time in EVID-OO3 is 04:13:52 pm UTC 2023 and the method used is the unix timestamp to readable UTC time conversion
B) The readable UTC time in EVID-OO4 is 00:00:00+00:00 and the method used is the MAC Absolute time method.

3. For byte-order values, state both interpretations and why they differ.

Answer:-

A) The interpretation for 725846400 is 978307200 & 2024-01-02 00:00:00+00:00
B) The interpretation for 78563412 is 305419896 & 2018915346

They differ because the endianness is ysed to determine the order in which bytes are interpreted.

Reflection: one paragraph on why number systems and timestamps matter in digital forensics.

The reason why number systems and timestamps matter in digital forensics is that with a strong knowledge in number system a forensic analyst can convert a data represented in a particular number system give insight on what that data represented and also with a knowledge on timestamp conversion it helps in knowing the actual time an event occur.

Conclusion

In conclusion the lab has add to my understanding on number system,ASCII,timestamp and also see how important they are in the field of digital forensics

Reference

➢ICDFA course Material (BVDF201(basic computer skills for digital forensics)- by COMMANDANT AMINU IDRIS

➢ICDFA Recorded sessions